2026-10-25 09:18:31 You

🟠 Level 8 — Secrets & Security

76. **Ansible Vault**
77. **Encrypting secrets**
78. **Vault passwords**
79. **Vault IDs**
80. **Secrets in variables**
81. **SSH keys**
82. **Become passwords**
83. **Avoiding secrets in Git**

ChatGPT gpt-6

Ansible Level 8 — Secrets & Security

Goal: Learn how to protect passwords, API tokens, SSH access, and privileged operations when automating servers with Ansible.

Core idea: Ansible playbooks and variable files often contain sensitive information. You must protect secrets at rest, control who can access them, and prevent them from appearing in Git repositories or logs.

76. Ansible Vault

Purpose: Ansible Vault encrypts sensitive files or individual strings so their contents are not stored as readable plaintext.

Common use cases:

- Database passwords
- Application credentials
- API tokens
- Cloud access credentials
- Sensitive variables used in deployments

Example project structure

```
ansible-project/
├── site.yml
├── inventory.ini
└── group_vars/
    └── production/
        └── vault.yml
```

Example plaintext variables before encryption:

```yaml
db_username: appuser
db_password: SuperSecret123
api_token: abc123secret
```

Create an encrypted file:

```bash
ansible-vault create group_vars/production/vault.yml
```

Ansible opens an editor and asks you to set a Vault password. After saving, the file contents are encrypted.

Useful commands

```bash
# View an encrypted file
ansible-vault view group_vars/production/vault.yml

# Edit an encrypted file
ansible-vault edit group_vars/production/vault.yml

# Encrypt an existing plaintext file
ansible-vault encrypt group_vars/production/vault.yml

# Decrypt a file back to plaintext
ansible-vault decrypt group_vars/production/vault.yml
```

Remember: Vault encrypts secrets; it does not automatically secure your entire server, protect exposed credentials, or replace access controls.

---

77. Encrypting Secrets

Purpose: Protect sensitive data by encrypting either an entire file or a single string.

A. Encrypt an entire file

Create `db_vars.yml`:

```yaml
db_username: appuser
db_password: MyDBPassword123
```

Encrypt it:

```bash
ansible-vault encrypt db_vars.yml
```

The contents become encrypted instead of readable YAML.

B. Encrypt a single string

```bash
ansible-vault encrypt_string \
  --name 'db_password' \
  'MyDBPassword123'
```

Ansible outputs an encrypted YAML variable that you can place in a variables file.

Example format:

```yaml
db_password: !vault |
          $ANSIBLE_VAULT;1.1;AES256
          <encrypted-content>
```

The encrypted content shown here is illustrative, not a working encrypted secret.

C. Use the encrypted file in a playbook

```yaml
- name: Configure database application
  hosts: web
```

```
  vars_files:
    - db_vars.yml

  tasks:
    - name: Display database username
      ansible.builtin.debug:
        var: db_username
```

Run the playbook with the Vault password prompt:

```
</> Bash

ansible-playbook site.yml --ask-vault-pass
```

Production example: Store production database credentials in an encrypted variables file and load them only when deploying the application.

Remember: Encrypting a file protects its stored contents. Once decrypted, secrets still need to be protected from logs, process output, and unauthorized access.

---

78. Vault Passwords

Purpose: Supply the password Ansible needs to decrypt Vault-encrypted data.

There are three common approaches.

A. Enter the password interactively

```
</> Bash

ansible-playbook site.yml --ask-vault-pass
```

- Ansible prompts you for the Vault password.
- Suitable for manual runs.
- The password does not need to be written into the playbook.

B. Read the password from a file

Create a password file outside your Git repository, with restrictive permissions.

```
</> Bash

chmod 600 ~/.ansible_vault_pass
```

Run:

```
</> Bash

ansible-playbook site.yml \
  --vault-password-file ~/.ansible_vault_pass
```

The file must contain the Vault password.

Security warning: Anyone who can read this file can decrypt the secrets protected by that password. Protect it with OS permissions and secure storage.

C. Use a password script or external secret manager

Ansible can use an executable password-client script that retrieves the password from an approved secret store.

Production example: A CI/CD pipeline retrieves a Vault password from its protected credentials store and supplies it to Ansible during deployment.

Remember: The Vault password must be protected separately from the encrypted files. If an attacker obtains both, the encrypted secrets can be exposed.

---

79. Vault IDs

Purpose: Manage encrypted files protected by different Vault passwords.

For example, development and production environments can use separate Vault passwords.

Environment	Vault ID	Password
Development	`dev`	Development Vault password
Production	`prod`	Production Vault password

Step 1 — Encrypt files using different IDs

```
</> Bash

ansible-vault encrypt \
  --vault-id dev@prompt \
  dev_secrets.yml
```

```
</> Bash

ansible-vault encrypt \
  --vault-id prod@prompt \
  prod_secrets.yml
```

Ansible prompts for the relevant passwords.

Step 2 — Use the files in a playbook

```
</> YAML

- name: Deploy application
  hosts: web
  vars_files:
    - dev_secrets.yml
    - prod_secrets.yml

  tasks:
    - name: Display deployment status
      ansible.builtin.debug:
        msg: "Deployment configuration loaded"
```

Run with the corresponding Vault IDs:

```
</> Bash

ansible-playbook site.yml \
  --vault-id dev@prompt \
  --vault-id prod@prompt
```

Ansible uses the available Vault passwords to decrypt the relevant data. Vault IDs are labels that help identify passwords; they are not a substitute for password security.

Production example: Separate development and production secrets so the team can manage different credentials and access policies.

Remember: Vault IDs help organize multiple Vault passwords. They do not automatically enforce environment access restrictions.

---

80. Secrets in Variables

Purpose: Use sensitive values in playbooks without exposing them in plaintext files or logs.

A. Store secrets in an encrypted variables file

`group_vars/production/vault.yml` :

```yaml
db_username: appuser
db_password: !vault |
  $ANSIBLE_VAULT;1.1;AES256
  <encrypted-content>
```

The encrypted value must be generated by Ansible Vault; the placeholder above is illustrative.

B. Use the secret in a task

```yaml
- name: Configure database application
  hosts: web
  vars_files:
    - group_vars/production/vault.yml

  tasks:
    - name: Run database migration
      ansible.builtin.command:
        argv:
          - /opt/myapp/migrate
      environment:
        DB_USERNAME: "{{ db_username }}"
        DB_PASSWORD: "{{ db_password }}"
      no_log: true
```

Important: `no_log: true` prevents Ansible from displaying the task's normal output and arguments in its logs. It does not guarantee that the application, operating system, or external logging systems cannot expose secrets.

C. Avoid printing secrets

Do not use this in production:

```yaml
- name: Print password
  ansible.builtin.debug:
    var: db_password
```

Even if the secret is encrypted at rest, debugging it can reveal its plaintext value.

Production best practices

- Encrypt sensitive variable files with Vault.
- Apply `no_log: true` to tasks that handle secrets.
- Restrict access to encrypted files and password sources.
- Prefer a dedicated secret manager for high-value production credentials.
- Rotate credentials when exposure is suspected.

Remember: Ansible variable names do not make values secure. A variable containing a password is still sensitive whether it is defined in a playbook, inventory, or another file.

---

81. SSH Keys

Purpose: Authenticate Ansible to remote Linux servers using SSH.

Ansible normally connects to Linux hosts over SSH. SSH key authentication is commonly preferred over entering an account password for every connection.

A. Generate an SSH key pair

On the Ansible control node:

```
</> Bash

ssh-keygen -t ed25519
```

This creates:

- **Private key:** Keep it secret.
- **Public key:** Can be copied to authorized remote hosts.

B. Install the public key on the remote server

```
</> Bash

ssh-copy-id ubuntu@192.168.1.10
```

You may be prompted for the remote user's password during the initial setup.

C. Configure Ansible inventory

`inventory.ini` :

```
</> INI
```

```
[web]
web01 ansible_host=192.168.1.10 ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/id_ed25519
```

D. Test connectivity

```
</> Bash

ansible web -i inventory.ini -m ansible.builtin.ping
```

If the SSH connection and Python prerequisites are correct, Ansible should return a successful result.

Production example: A deployment pipeline connects to approved application servers using a protected SSH private key.

Security rules

- Never commit private SSH keys to Git.
- Restrict access to private key files.
- Prefer managed SSH certificates or short-lived credentials where supported.
- Avoid sharing one personal private key across an entire team.
- Use a dedicated automation account with only the permissions it needs.

Remember: The public key goes on the server; the private key stays protected on the control node or in an approved credential system.

---

82. Become Passwords

Purpose: Authenticate when Ansible needs elevated privileges, such as running tasks with `sudo`.

Example:

```
</> YAML

- name: Install Nginx
  hosts: web
  become: true

  tasks:
    - name: Install Nginx
      ansible.builtin.apt:
        name: nginx
```

```
        state: present
```

Here, `become: true` asks Ansible to use privilege escalation.

If the remote account requires a `sudo` password, use:

```bash
ansible-playbook site.yml --ask-become-pass
```

Ansible prompts for the privilege-escalation password.

SSH password vs Become password

Password	Purpose	Typical option
SSH connection password	Authenticate the remote login	`--ask-pass`
Become password	Obtain elevated privileges, commonly via `sudo`	`--ask-become-pass`
Vault password	Decrypt Vault-encrypted data	`--ask-vault-pass`

These passwords serve different purposes.

Production example: Ansible logs in as a restricted deployment user, then uses controlled privilege escalation for installing packages and changing system configuration.

Remember: `become: true` enables privilege escalation; it does not itself supply a password or grant the account permission to use `sudo`.

For automation, teams often configure narrowly scoped passwordless `sudo` for a dedicated service account, subject to their security policies. Avoid granting unrestricted root access unnecessarily.

---

83. Avoiding Secrets in Git

Purpose: Prevent passwords, API keys, private keys, and other credentials from being exposed through source control.

A. Never commit plaintext secrets

Bad project structure:

```
ansible-project/
├── site.yml
├── inventory.ini
└── passwords.yml
```

If `passwords.yml` contains plaintext credentials and is committed, everyone with repository access may be able to read them.

Better structure:

```
ansible-project/
├── site.yml
├── inventory.ini
├── group_vars/
│   └── production/
│       └── vault.yml
└── .gitignore
```

Encrypt `vault.yml` with Ansible Vault before committing it.

B. Add sensitive files to `.gitignore`

```
.env
*.pem
*.key
vault-password.txt
```

This helps prevent accidental addition of common secret files, but `.gitignore` does not protect files already tracked by Git.

C. Never hardcode credentials in playbooks

Avoid:

```yaml
vars:
  api_password: "MyRealPassword123"
```

Instead, use a Vault-encrypted variable or retrieve the secret from a suitable secret manager.

D. Check what Git will commit

```bash
git status
git diff --cached
```

Use secret-scanning tools in your repository or CI/CD pipeline to detect accidental credential exposure.

E. If a secret is committed accidentally

1. Revoke or rotate the exposed credential immediately.
2. Check where the credential may have been accessed or used.
3. Remove the secret from the current files and, where appropriate, clean repository history.
4. Add preventive controls such as secret scanning and protected credential storage.

Important: Deleting a secret in a later commit does not remove it from earlier Git history. Treat exposed credentials as compromised.

Quick revision — Level 8

#	Concept	Remember this
76	Ansible Vault	Encrypt sensitive data
77	Encrypting secrets	Encrypt complete files or individual strings
78	Vault passwords	Supply the key needed to decrypt Vault data
79	Vault IDs	Identify and manage multiple Vault passwords
80	Secrets in variables	Use protected variables and avoid leaking output
81	SSH keys	Authenticate the control node to remote hosts
82	Become passwords	Authenticate privilege escalation
83	Avoiding secrets in Git	Encrypt, exclude, scan, and rotate credentials

Minimal hands-on practice

- [ ] Create a `vault.yml` containing a test password and encrypt it.
- [ ] Load the encrypted file in a playbook and run with `--ask-vault-pass`.
- [ ] Encrypt a single variable with `ansible-vault encrypt_string`.
- [ ] Configure SSH key authentication and test with the Ansible `ping` module.
- [ ] Run a privileged task using `--ask-become-pass` on a test server that requires it.
- [ ] Add sensitive local files to `.gitignore` and verify that an encrypted Vault file can be committed without exposing its contents.

Interview priority: Understand Vault vs Vault IDs, SSH password vs become password vs Vault password, how encrypted variables are used, and how to prevent secrets from leaking through Git and Ansible logs.